Esercitazione: tabellina, conto alla rovescia, contatore e somma

Lezione 15 — 1IOT, SST-ICT-01 — Quattro programmi con il controllo del numero e il ciclo for

02/11/2026

1 Che cosa abbiamo fatto oggi

Oggi non c'è un argomento nuovo: è stata una lezione di **esercizi**, tutti scritti su **onlinegdb**. Abbiamo fatto **quattro programmi**, due insieme e due prima da soli e poi corretti insieme:

1. la **tabellina** di un numero (Sezione 2);
2. il **conto alla rovescia** (Sezione 3);
3. **quante volte si ripete** un ciclo da partenza ad arrivo: il **contatore** (Sezione 5);
4. i **multipli** di un numero fino a 100 e la loro **somma** (Sezione 6).

In mezzo abbiamo parlato di **intelligenza artificiale** (Sezione 4) e, alla fine, di **come si studia** (Sezione 8).

Tutti e quattro i programmi hanno la **stessa forma**: prima **chiedo un numero** e controllo che sia giusto, poi uso un **ciclo for**, poi **stampo il risultato**. Se capisci questa forma, gli esercizi del compito per casa sono la stessa cosa con numeri diversi.

2 Primo programma: la tabellina

Esempio

La richiesta (Compito C02, Esercizio 1). Scrivi un programma che chieda un numero da 1 a 10 (se è sbagliato, la domanda si ripete) e stampi, su una riga, la sua tabellina: da `n` fino a 10 volte `n`.

2.1 Controllare il numero: il do while

Mi serve una variabile `int n`. La domanda va fatta **almeno una volta** e va **ripetuta** se il numero è sbagliato: quindi uso il **do while**.

```c
do
{
    printf("Inserisci un numero da 1 a 10: ");
    scanf("%d", &n);
} while ((n < 1) || (n > 10));
```

Attenzione

Lo scanf si sbaglia spesso. Correggendo i compiti, due errori tornano continuamente:

- manca la **virgola** dopo le virgolette: `scanf("%d" &n)` è sbagliato;
- le **virgolette** si chiudono subito dopo la `d`, non in fondo: `scanf("%d, &n")` è sbagliato.

La forma giusta è una sola: `scanf("%d", &n);`

2.2 Quando è sbagliato? Il disegno sulla retta

Quando scrivo la condizione mi aiuta un **disegno**. Disegno una **retta** e ci metto sopra i numeri: lo 0, a destra 1, 2, 3 e così via, a sinistra -1, -2, -3. Poi coloro la zona dei numeri **validi** (da 1 a 10) e le zone dei numeri **sbagliati**.

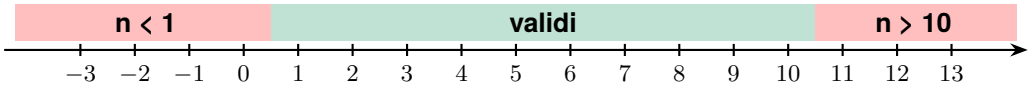


Figura 1: La retta dei numeri: la domanda si ripete quando il numero cade in una delle due zone rosse.

La domanda si ripete quando sono sulle zone **rosse**: quando `n` è **minore di 1** **oppure** `n` è **maggiore di 10**. "Oppure" si scrive `||`, ed ecco la condizione: `(n < 1) || (n > 10)`. Le parentesi attorno ai due pezzi **non sono obbligatorie**: si mettono per essere più chiari.

- **Attenzione al ragionamento**: nel do while scrivo quando **ripetere**, cioè quando il numero è **sbagliato**. Se scrivessi `(n >= 1) && (n <= 10)`, che descrive la zona verde, il programma ripeterebbe la domanda proprio quando il numero è giusto. Con `&&` ("e") si descrive una zona **compresa** tra due valori; con `||` ("oppure") le due zone **esterne**.
- Va bene anche `n <= 0` al posto di `n < 1`: con i numeri **interi** è la stessa cosa. Non lo sarebbe più con i numeri con la virgola: 0,99 è minore di 1 ma non è minore o uguale a 0.
- Questo modo di disegnare le zone su una retta lo ritroverete in matematica con le **disequazioni**.

2.3 Prima di scrivere il for: un esempio con i numeri

Dopo il do while c'è una riga `printf("\n");`. **Non è necessaria**: quando premi Invio il cursore va già a capo. Aggiunge solo una riga vuota, per una questione di impaginazione. Non è sbagliata e la lasciamo.

Prima di scrivere il ciclo facciamo un esempio, per capire **qual è la regola**. Se `n` vale 2, voglio vedere questi numeri:

Numero stampato	2	4	6	8	10	12	14	16	18	20
Come si ottiene	$2 \cdot 1$	$2 \cdot 2$	$2 \cdot 3$	$2 \cdot 4$	$2 \cdot 5$	$2 \cdot 6$	$2 \cdot 7$	$2 \cdot 8$	$2 \cdot 9$	$2 \cdot 10$
Valore di `i`	1	2	3	4	5	6	7	8	9	10

Il numero che cambia è il secondo: serve una **nuova variabile**, `i`, che parte da **1** e arriva a **10** compreso. Il numero da stampare è sempre `n * i`.

2.4 Trovare un errore ragionando

Proviamo a scrivere il limite del ciclo così: se `n` vale 2 voglio arrivare a 20, se vale 7 a 70, quindi `i <= n * 10`. Sembra giusto. Proviamo.

```c
for (int i = 1; i <= n * 10; i++)
{
    printf("%d ", n * i);
}
```

```
Inserisci un numero da 1 a 10: 15
Inserisci un numero da 1 a 10: 12
Inserisci un numero da 1 a 10: -6
Inserisci un numero da 1 a 10: 2

2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40
```

Il controllo del numero **funziona**: 15, 12 e -6 vengono rifiutati. Ma i numeri arrivano a **40** invece che a 20. Dove sta l'errore? Si ragiona sui dati:

- con `n` = 2 stampa **20** numeri invece di 10, cioè il doppio;
- con `n` = 3 ne stampa 30, cioè il triplo, e arriva a 90;
- con `n` = 10 ne stampa 100, e arriva a 1000.

Stampa **n volte** più numeri del dovuto: il colpevole è quel `n * 10`. Riguardiamo la tabella: `i` va **da 1 a 10** per qualunque valore di `n`. Il limite è semplicemente **10**. Non dipende da `n`: è il risultato `n * i` che dipende da `n`.

Ecco perché le altre correzioni che vengono in mente non vanno bene:

- `i <= 5` funziona solo per il 2: è la metà di 10 solo perché stiamo guardando il 2, e con il 3 stamperebbe solo fino a 15;
- `i <= i * 10` non finisce mai: `i` cambia a ogni giro, e il limite si sposta insieme a lei;
- `i += 2` ("perché sono i pari") non serve: i pari vengono fuori dalla moltiplicazione `n * i`, mentre `i` deve contare 1, 2, 3.

Nota

Un programma sbagliato **non è una tragedia**. Se trovo un errore **torno sui miei passi** e capisco **dove** ho sbagliato il ragionamento. Scrivere una tabellina con i numeri che mi aspetto aiuta molto: qui mostra subito che `i` va sempre da 1 a 10. Non basta dire "sbagliato": bisogna capire **che cosa** non va, e provarlo.

2.5 Il programma completo

```c
#include <stdio.h>

int main()
{
    int n;

    do
    {
        printf("Inserisci un numero da 1 a 10: ");
        scanf("%d", &n);
    } while ((n < 1) || (n > 10));

    printf("\n");

    for (int i = 1; i <= 10; i++)
    {
        printf("%d ", n * i);
    }

    return 0;
}
```

```
Inserisci un numero da 1 a 10: 2

2 4 6 8 10 12 14 16 18 20
```

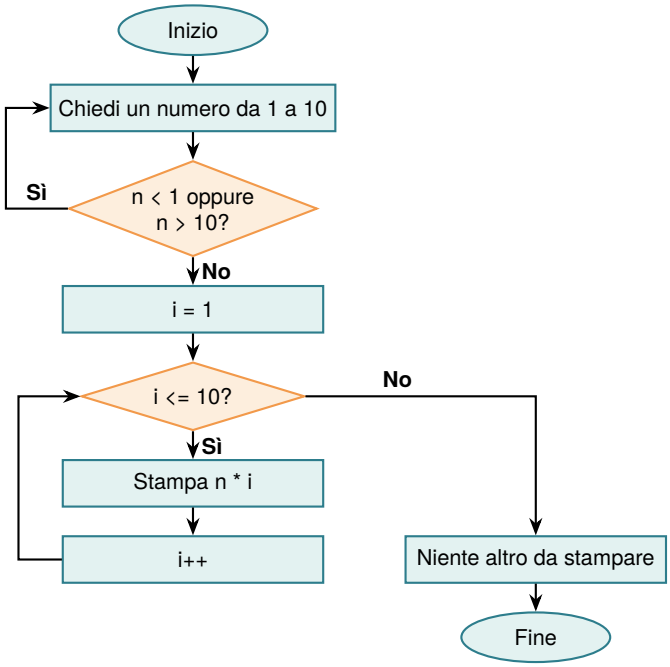


Figura 2: Diagramma di flusso della tabellina.

2.6 Che cosa fa davvero il printf

Il printf **non è una formalità**: è lui che **mette sullo schermo** i risultati. Se tolgo il `printf` dal ciclo, il programma fa gli stessi giri ma **non si vede niente**.

Il printf decide anche **come** appaiono i dati, cioè li **formatta**. Quello che scrivo tra le virgolette finisce sullo schermo: se dentro scrivo `ciao`, a ogni giro compare `ciao`.

- `printf("%d", n * i);` attacca i numeri tutti di fila: `2468101214...`;
- `printf("%d ", n * i);` lascia uno **spazio** dopo ogni numero: `2 4 6 8 ...`;
- `printf("%d\n", n * i);` va **a capo** dopo ogni numero: li mette in **colonna**.

E se vogliamo vedere anche il calcolo, `2 * 1 = 2`? In un printf si possono mettere **più valori**: ogni `%d` rappresenta il **numero contenuto in una variabile** o il risultato di un calcolo, e i valori dopo la virgola prendono il posto dei `%d` **nell'ordine** in cui sono scritti. Il primo `%d` è `n`, il secondo è `i`, il terzo è `n * i`.

```c
for (int i = 1; i <= 10; i++)
{
    printf("%d * %d = %d\n", n, i, n * i);
}
```

```
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
...
2 * 10 = 20
```

Si può ottenere lo stesso risultato con **tre printf** diversi, mettendo il `\n` solo sull'ultimo:

```c
for (int i = 1; i <= 10; i++)
{
    printf("%d * ", n);
    printf("%d = ", i);
    printf("%d\n", n * i);
}
```

Attenzione: questa è una **variante** per capire il printf. La richiesta dell'esercizio diceva "**su una riga**", quindi la versione da consegnare è quella del programma completo qui sopra.

3 Secondo programma: il conto alla rovescia

Esempio

La richiesta. Chiedi all'utente di inserire un numero da 1 a 20 (effettua il controllo sul corretto inserimento). Stampa tutti i numeri da quello inserito fino a 1, sulla stessa riga. Alla fine stampa a capo "Via!".

Questo l'avete fatto **prima da soli**. Assomiglia molto al programma di prima: il do while è lo stesso, cambiano solo i numeri. Il for invece è nuovo, perché deve **scendere**. Prendiamo un esempio con `n` = 12: devo stampare 12, 11, 10, ... fino a 1, e poi a capo `Via!`.

I tre pezzi del for si ricavano così:

- **da dove parte**: dal numero inserito. Il 12 è solo un esempio e il numero sta in `n`, quindi `int i = n`, non `int i = 12`;

- **fino a quando continua**: finché `i >= 1`. Deve stampare anche l'1, quindi "maggiore o uguale";
- **come cambia**: come `i++` fa salire di 1, così **`i--`** (il **decremento**) fa scendere di 1.

Attenzione

La condizione del for non è il punto in cui ci si ferma: è la condizione per cui si continua. Quando il ciclo sale scriviamo di solito `i <= n`. Quando scende il segno si **rovescia**: `i >= 1`. Se scrivessi `i <= 1`, con 12 la condizione sarebbe falsa subito e il ciclo **non partirebbe nemmeno**. Se scrivessi `i > 1`, l'1 non verrebbe stampato.

```c
#include <stdio.h>

int main()
{
    int n;

    do
    {
        printf("Inserisci un numero da 1 a 20: ");
        scanf("%d", &n);
    } while ((n < 1) || (n > 20));

    for (int i = n; i >= 1; i--)
    {
        printf("%d ", i);
    }

    printf("\nVia!");

    return 0;
}
```

```
Inserisci un numero da 1 a 20: 12
12 11 10 9 8 7 6 5 4 3 2 1
Via!
```

Il `\n` prima di `Via!` non è un dettaglio: **la richiesta diceva "a capo"**. Senza, `Via!` finisce attaccato all'1. Il printf di `Via!` sta **fuori dal ciclo**, perché deve comparire una volta sola. **Leggete bene la consegna**: quello che non viene fatto come richiesto viene contato come errore.

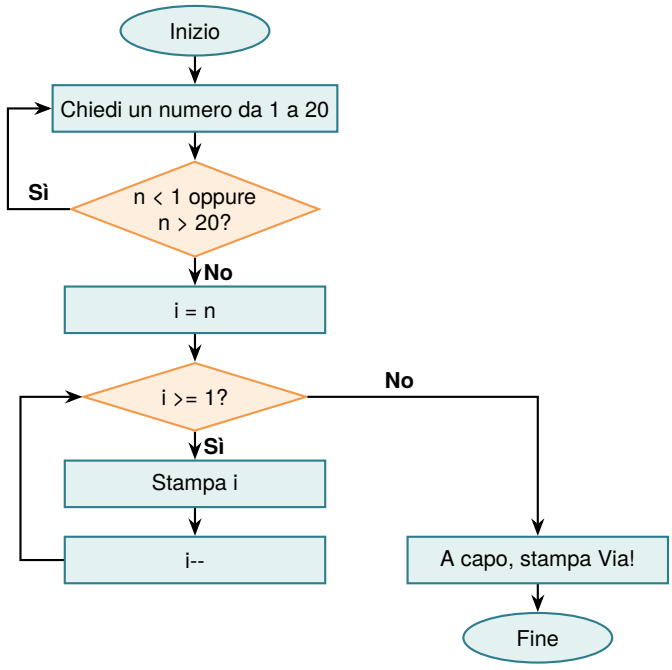


Figura 3: Diagramma di flusso del conto alla rovescia.

4 L'intelligenza artificiale: perché non per fare i compiti

Quello che consegnate viene **controllato**, anche per capire se è stato scritto con l'intelligenza artificiale, e si riconosce. Ma il punto non è questo. Non sto dicendo che **non si deve usare**: sarebbe poco realistico, ormai è dappertutto, ed è giusto che sia così. Il problema è che **bisogna imparare a usarla**.

ChatGPT, Claude, Perplexity e gli altri sono **modelli matematici**: programmi che lavorano con la **statistica**, cioè con le **probabilità**.

Esempio

Il dado. Tiro un dado: la probabilità che esca il 4 è

$$\frac{1}{6} \approx 16{,}7\%$$

Se lo tiro due volte, la probabilità che esca il 4 **tutte e due le volte** è **più bassa**, non più alta:

$$\frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36} \approx 2{,}8\%$$

È più alta solo la probabilità che esca **almeno una volta** in due tiri:

$$1 - \frac{5}{6} \cdot \frac{5}{6} = \frac{11}{36} \approx 30{,}6\%$$

Un assistente di AI **legge** quello che scrivete e **calcola** qual è la risposta più probabile. Ma sapere che il 4 esce una volta su sei **non garantisce** che esca proprio adesso. Allo stesso modo, che una risposta sia la più probabile **non garantisce che sia giusta**.

- Finché ci giochi, nessun problema. Se prendi **decisioni importanti** sulla base di quello che dice, metti in conto che prima o poi **un errore arriva**.
- Se l'errore si paga, lo paghi **tu**: la macchina non ha responsabilità. "Me l'ha detto l'AI" non è una giustificazione.
- Io la uso moltissimo, ma **so già** queste cose e le ho studiate: mi serve per **velocizzare** il lavoro, e capisco quando sbaglia.
- Voi siete qui per impararle: vi manca ancora la **conoscenza** per capire se quello che dice è giusto o sbagliato. Se scrive lei il programma, **non imparate**. Prima si impara, poi si usa a pieno.

Nota

Chiedere all'AI **non è vietato**, purché poi si capisca il **perché e il per come**. Un assistente sa molte più cose di noi e spesso propone soluzioni **mai viste in classe**: se in un compito compare una cosa così, è chiaro che non viene da te. Se ti blocchi, mandale il tuo codice e scrivi: **"non darmi il codice, spiegami"**. La modifica poi la fai tu. Quando andrete a lavorare la userete: ma dovrete saper correggere quando dice una sciocchezza.

5 Terzo programma: quante volte si ripete? Il contatore

Esempio

La richiesta (Compito C02, Esercizio 2). Chiedi all'utente il numero di partenza, il numero di arrivo e il passo (maggiore di 0, controllato). Stampa i numeri dalla partenza fino a prima dell'arrivo e, alla fine, quante volte ha ripetuto.

5.1 Che cos'è il passo

Il **passo** è **di quanto vado avanti** ogni volta. Se devo andare da 1 a 10 e vado avanti di 1 in 1, il passo è 1. Se vado avanti 2, 4, 6, 8, 10, il passo è 2. All'utente chiedo tre valori: **da dove** comincio a contare, **dove** devo finire e il **passo**.

"Fino a **prima** dell'arrivo" vuol dire che l'arrivo **non si stampa**: da 1 a 10 con passo 1 stampo da 1 a 9. Quindi nel for la condizione è `i < arrivo`, non `i <= arrivo`.

5.2 Dichiarare più variabili insieme

Mi servono tre variabili. Si possono dichiarare in due modi, e sono **la stessa cosa**: scegli quella che ti piace di più.

```c
int partenza;
int arrivo;
int passo;

int partenza, arrivo, passo;
```

5.3 Che cosa va controllato

Su partenza e arrivo **non ci sono controlli**: va bene qualunque numero, quindi basta printf e scanf. Il **passo** invece deve essere **maggiore di 0**: se devo controllarlo, serve il **do while**, e il do while si ripete quando il numero è **sbagliato**. Sbagliato è **minore o uguale a 0**:

```c
do
{
    printf("Inserisci il passo maggiore di 0: ");
    scanf("%d", &passo);
} while (passo <= 0);
```

Attenzione

Perché anche lo 0 è sbagliato. Con il passo a 0, `i` non cambia mai: la condizione `i < arrivo` resta sempre vera e il ciclo **non finisce mai**. Per questo nel controllo si scrive `passo <= 0` e non solo `passo < 0`.

5.4 Il for con il passo

Il for parte dalla partenza (`int i = partenza`), continua finché `i < arrivo` e a ogni giro aggiunge il passo: `i += passo`. Non si può usare `i++`, che aumenta sempre di 1. Se `i` vale 3 e il passo è 2, al giro dopo `i` vale 5, poi 7, poi 9, 11, 13, 15.

Nel printf lasciamo uno **spazio** dopo `%d`, così i numeri si leggono meglio.

Nota

Controlla le variabili una per una. Quando copi le righe del printf e dello scanf è facile lasciare il nome sbagliato: se in una riga c'è scritto `&arrivo` dove serviva `&partenza`, il programma parte ma i conti non tornano. Se i numeri sono strani, rileggi queste righe per prime.

5.5 Il contatore

Non voglio solo vedere i numeri: voglio sapere **quante volte** ha girato il ciclo. Serve una nuova variabile, il **contatore**, che chiamiamo `cicli`. Le regole sono tre:

- **prima** del ciclo vale 0: `cicli = 0`;
- **dentro** il ciclo, ogni volta che stampo qualcosa, aumenta di 1: `cicli++`;
- **dopo** il ciclo lo stampo, una volta sola.

```c
#include <stdio.h>

int main()
{
    int partenza, arrivo, passo;
    int cicli;
    cicli = 0;

    printf("Inserisci il numero di partenza: ");
    scanf("%d", &partenza);

    printf("Inserisci il numero di arrivo: ");
    scanf("%d", &arrivo);

    do
    {
        printf("Inserisci il passo maggiore di 0: ");
        scanf("%d", &passo);
    } while (passo <= 0);

    for (int i = partenza; i < arrivo; i += passo)
    {
        printf("%d ", i);
        cicli++;
    }

    printf("\nAbbiamo ripetuto %d volte", cicli);

    return 0;
}
```

```
Inserisci il numero di partenza: 3
Inserisci il numero di arrivo: 17
Inserisci il passo maggiore di 0: 3
3 6 9 12 15
Abbiamo ripetuto 5 volte
```

Prova anche i casi strani. Se la partenza è più grande dell'arrivo (per esempio da 4 a 1, con passo 9), la condizione `i < arrivo` è falsa **subito**: il ciclo non parte, non stampa niente e il contatore resta a 0. Non è un errore: è esattamente quello che abbiamo chiesto.

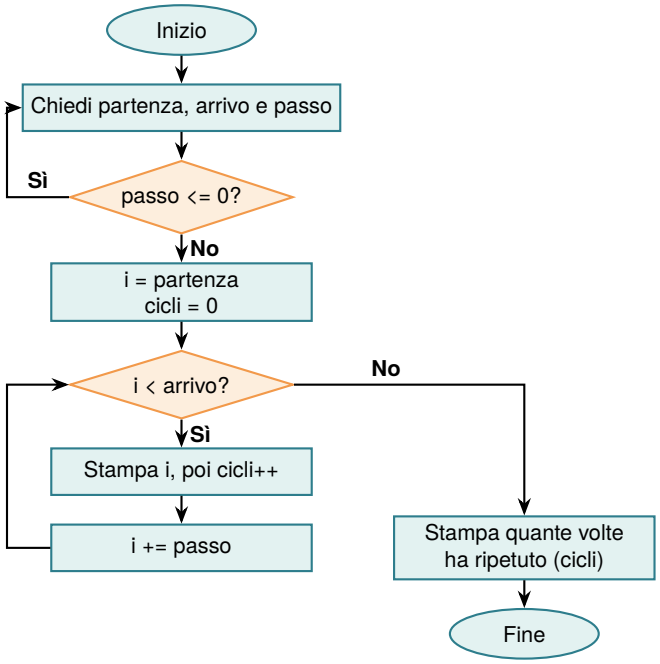


Figura 4: Diagramma di flusso del contatore. Il do while controlla solo il passo: partenza e arrivo si chiedono prima, senza controlli.

Nota

I commenti. Nel codice si può scrivere qualunque cosa dopo le due barre `//`: è un **commento**, e la macchina lo ignora. Il programma funziona uguale anche senza. I commenti servono **a chi legge**, per capire che cosa sta facendo il programma.

6 Quarto programma: i multipli e la somma

Esempio

La richiesta. Chiedi un numero da 2 a 9 (fai il controllo che sia corretto). Stampa i multipli del numero fino a 100 e alla fine stampa la somma di tutti quei numeri. Per esempio, se inserisco 7, stamperà 7 14 21 ... 98 e poi la somma, che fa 735.

Anche questo l'avete provato **prima da soli**. Il do while è sempre lo stesso: si ripete finché `(n < 2) || (n > 9)`.

6.1 I multipli

Con 7 voglio i numeri 7, 14, 21, 28… Nel for:

- **parte dal numero**: `int i = n;`
- **continua fino a 100** compreso: `i <= 100;`
- ogni volta **aggiunge il numero**: `i += n`.

Attenzione

Si aumenta **`i`, non `n`**. Scrivere `n += i` sarebbe sbagliato: `n` è il numero scelto dall'utente e deve **restare fermo**. È `i` che vale 7, poi 14, poi 21. E ricordate l'**ordine**: il for si scrive **dopo** il do while e non in un punto qualunque, perché prima serve il numero.

6.2 La somma: l'accumulatore

Voglio la somma di tutti i numeri stampati. Mi serve una variabile `somma`, che **all'inizio vale 0**: è il punto di partenza. Ogni volta che stampo `i`, lo aggiungo:

```
somma += i;
```

che è la stessa cosa di `somma = somma + i`. Con 7 succede questo:

`i`	7	14	21	28	35	…
`somma` dopo la riga	7	21	42	70	105	…

Il valore **si accumula**. Alla fine del ciclo, **fuori** dalle graffe, stampo il risultato.

```c
#include <stdio.h>

int main()
{
    int n;
    int somma;
    somma = 0;

    do
    {
        printf("Inserisci un numero da 2 a 9: ");
        scanf("%d", &n);
    } while ((n < 2) || (n > 9));

    for (int i = n; i <= 100; i += n)
    {
        printf("%d ", i);
        somma += i;
    }

    printf("\nLa somma di tutti i numeri vale %d", somma);

    return 0;
}
```

```
Inserisci un numero da 2 a 9: 7
7 14 21 28 35 42 49 56 63 70 77 84 91 98
La somma di tutti i numeri vale 735
```

Provate sempre il programma con l'esempio della richiesta: se mettendo 7 non ottieni 735, c'è qualcosa da sistemare. Se ti dimentichi la riga `somma += i`, per esempio, il risultato è 0.

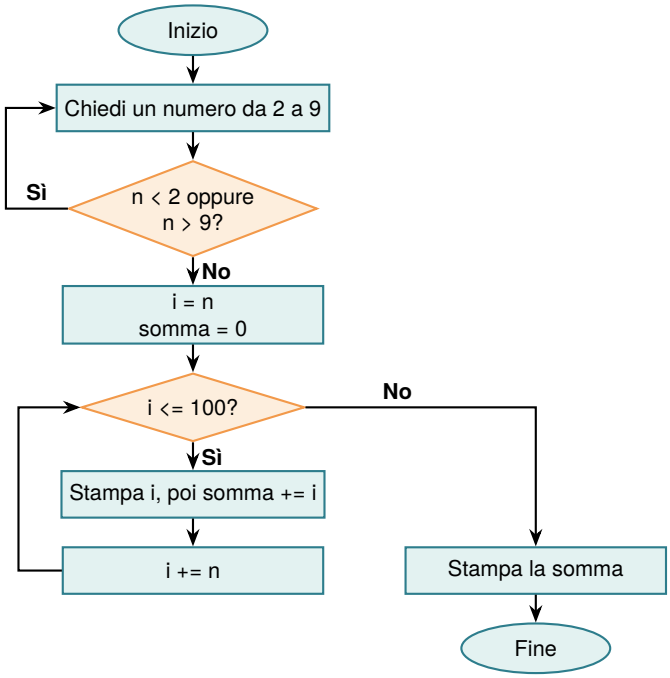


Figura 5: Diagramma di flusso dei multipli con la somma.

6.3 Contatore e somma a confronto

	Contatore	Somma (accumulatore)
A che cosa serve	quante volte ha girato	quanto fanno i valori messi insieme
Prima del ciclo	`cicli = 0`	`somma = 0`
Dentro il ciclo	`cicli++` (sempre 1 in più)	`somma += i` (il valore di adesso)
Dopo il ciclo	si stampa	si stampa

7 Riepilogo: i for di oggi

Programma	Il for	Che cosa fa
Tabellina	`for (int i = 1; i <= 10; i++)`	`i` conta sempre da 1 a 10
Conto alla rovescia	`for (int i = n; i >= 1; i--)`	scende da `n` a 1
Contatore	`for (int i = partenza; i < arrivo; i += passo)`	salta di `passo`, arrivo escluso
Multipli	`for (int i = n; i <= 100; i += n)`	salta di `n`, 100 compreso

Prima di scrivere un for, rispondi a tre domande: **da dove parto?** Finché che cosa **continuo?** Come **cambio** a ogni giro?

8 Come si studia, e che cosa succede adesso

In tre ore abbiamo fatto **quattro esercizi**: sono pochi, il programma della mattina ne prevedeva dodici, sei fatti da me e sei da voi. Si sta imparando e ci mancherebbe altro, ma così **non ne veniamo fuori**. Vorrei passare a cose più interessanti e più pratiche, come **Arduino**, ma per farlo bisogna essere **allineati** sulla scrittura di un programma. Quindi:

- **Studiare non vuol dire** scorrere la dispensa. Vuol dire leggerla **dall'inizio alla fine**, riprendere in mano gli **esercizi fatti in classe** e **provare a rifarli da soli**.
- I **compiti per casa** si consegnano. Sono il modo in cui riesco a capire **dov'è il problema**: se inizi bene ma sbagli il for, so che dobbiamo lavorare sul for. Se il programma lo scrive un'AI, non so che cosa fare per aiutarti.

Nota

Per la prossima volta: Compito C03, un blocco di esercizi su for, contatore e somma, da consegnare su Classroom. Lo consegnano **tutti**: quello che riesci a fare, come riesci a farlo, senza farti **scrivere il codice** dall'AI.
Lunedì prossimo vorrei provare un esperimento: una **sfida a coppie**, con un **pilota** che scrive e un **navigatore** che tiene le mani in tasca e gli dice che cosa fare, come in macchina.